Manual Completo de Ciência e Análise de Dados em Python

Guia Prático de Referência: Análise Estruturada, Ferramentas do OS e Ecossistema Data Science

1. Introdução à Análise de Dados, Lógica Básica e Tratamento

A **Análise de Dados** é o processo sistemático de aplicar técnicas estatísticas e lógicas para avaliar, limpar, transformar e modelar dados com o objetivo de extrair insights úteis e apoiar a tomada de decisões estratégicas.

O Ciclo Fundamental do Tratamento de Dados:

- **Coleta e Carga:** Importação de fontes brutas (arquivos CSV, bancos de dados, APIs).
- **Limpeza e Higienização:** Identificação de dados ausentes (`NaN`), remoção de duplicatas e correção de inconsistências de digitação ou formatação.
- **Transformação e Tipagem:** Garantir que cada coluna possua o tipo de dado correto (texto, inteiros, decimais, datas).
- **Lógica e Regras de Negócio:** Aplicação de estruturas condicionais e ordenações para responder a perguntas específicas sobre o negócio.
- **Exploração e Visualização:** Uso de estatística descritiva e gráficos para identificar tendências, padrões e *outliers*.

2. Pandas (Manipulação e Análise de Dados)

O **Pandas** é a biblioteca principal para estruturação, limpeza e transformação de tabelas (DataFrames). Cada coluna de um DataFrame é tratada como uma `Series`.

Função / Método	Descrição e Aplicação
`pd.read_csv("caminho.csv")`	Carrega arquivos CSV em um DataFrame.
`df.head() / df.info()`	Exibe as primeiras linhas e o resumo da estrutura/tipos de dados.
`df['coluna'].value_counts()`	Conta a frequência de cada categoria única em uma coluna.
`df.query("nota <= 2")`	Filtra linhas da tabela com base em uma condição lógica expressa como string.
`df.groupby('cat')['val'].mean()`	Agrupa por categorias e calcula métricas agregadas (ex: média salarial por setor).
`df.sort_values(by='col', ascending=False)`	Reordena a tabela. *Nota: Requer reatribuição (`df = df.sort_values(...)`) para salvar.*
`pd.crosstab(df['a'], df['b'])`	Gera uma tabela cruzada de contagem de frequência entre duas variáveis categóricas.

3. NumPy (Computação Numérica e Vetorização)

O **NumPy** provê suporte a `arrays` n-dimensionais de alta performance. Suas operações são vetorizadas, eliminando a necessidade de laços `for` manuais.

- **Operações Vetorizadas:** Cálculos aplicados diretamente ao array de uma só vez (ex: `array * 0.9` aplica 10% de desconto a todos os itens).
- **Métricas Estatísticas:** Funções ultrarrápidas como `np.mean()`, `np.median()`, `np.std()`, `np.max()` e `np.min()`.
- **Filtros Condicionais em Tabelas (`np.where`):** Funciona como a regra SE/IF.

```
# Exemplo de criação de coluna condicional no Pandas usando NumPy:
import numpy as np

tabela['status'] = np.where(tabela['faturamento'] > 6000, 'Alto', 'Padrão')

# Para múltiplas condições (3 ou mais):
regras = [tabela['vendas'] >= 10000, tabela['vendas'] >= 5000, tabela['vendas'] < 5000]
categorias = ['Super Pico', 'Médio', 'Baixo']
tabela['categoria'] = np.select(regras, categorias, default='Outro')
```

4. Seaborn (Visualização Estatística)

Construída sobre o Matplotlib, a **Seaborn** é especializada em gráficos estatísticos elegantes com poucas linhas de código.

Função	Finalidade do Gráfico
`sns.heatmap(matriz.corr(), annot=True)`	Mapa de calor para matrizes de correlação (-1 a +1). Mostra a força da relação entre variáveis.
`sns.boxplot(data=df, x='cat', y='val')`	Exibe mediana, quartis e outliers agrupados por categorias.
`sns.scatterplot(data=df, x='x', y='y', hue='cat')`	Gráfico de dispersão entre duas colunas numéricas, agrupado por cor via `hue`.
`sns.regplot(data=df, x='x', y='y')`	Gráfico de dispersão incluindo a linha de tendência estatística (regressão linear).
`sns.histplot(data=df, x='val', kde=True)`	Histograma com curva de densidade de frequência da variável.

5. Matplotlib (Motor Gráfico e Customização)

É a infraestrutura gráfica base do Python. O comando `plt.show()` renderediza o painel ativo na memória do programa.

> 💡 **Por que a Seaborn usa plt.show()?**
> A Seaborn desenha os elementos no "painel ativo" gerenciado pela Matplotlib. Por isso, ao criar um `sns.heatmap()`, deve-se chamar `plt.show()` para exibir a janela final.

- `plt.figure(figsize=(10, 5))`: Define a largura e altura da prancheta gráfica.
- `plt.title() / plt.xlabel() / plt.ylabel()`: Personaliza título e rótulos dos eixos.
- `plt.subplots(linhas, colunas)`: Permite desenhar múltiplos gráficos lado a lado na mesma janela.
- `plt.savefig('grafico.png')`: Exporta a imagem gerada para o disco.

6. Plotly Express (Visualização Interativa de Dados)

O **Plotly Express** (`import plotly.express as px`) gera gráficos interativos modernos em HTML, permitindo zoom, navegação hover (ao passar o mouse) e isolamento de categorias.

Tipo de Gráfico	Comando no Plotly Express	Função e Casos de Uso
Barras	`px.bar(df, x='x', y='y', color='c')`	Comparar valores entre categorias ou subgrupos.
Linhas	`px.line(df, x='data', y='valor')`	Acompanhar evolução e tendências de séries temporais.
Dispersão (Scatter)	`px.scatter(df, x='x', y='y', color='c')`	Analisar correlação e distribuição entre duas variáveis numéricas.
Boxplot	`px.box(df, x='cat', y='val')`	Visualizar a variação, mediana, quartis e outliers de um grupo.
Histograma	`px.histogram(df, x='coluna')`	Mostrar a distribuição de frequência de dados numéricos.
Setores (Pizza)	`px.pie(df, names='cat', values='val')`	Mostrar proporção percentual da composição das partes de um todo.
Rosca (Donut)	`px.pie(df, names='cat', values='v', hole=0.4)`	Proporção visual moderna mantendo leitura amigável do centro.
Área	`px.area(df, x='data', y='valor')`	Volume cumulativo de métricas ao longo do tempo.
Mapa de Calor	`px.imshow(df.corr())`	Visualizar matrizes de correlação e superfícies 2D.
Treemap	`px.treemap(df, path=['c1', 'c2'], values='v')`	Hierarquia em blocos proporcionais retangulares.
Sunburst	`px.sunburst(df, path=['c1', 'c2'], values='v')`	Gráfico de pizza multinível para estruturas hierárquicas.
Violino	`px.violin(df, x='cat', y='val', box=True)`	Boxplot combinado com densidade de probabilidade.

Tipo de Gráfico	Comando no Plotly Express	Função e Casos de Uso
Dispersão 3D	`px.scatter_3d(df, x='x', y='y', z='z')`	Análise tridimensional interativa entre três variáveis.
Linhas 3D	`px.line_3d(df, x='x', y='y', z='z')`	Trajetórias temporais ou espaciais em espaço tridimensional.
Funil (Funnel)	`px.funnel(df, x='val', y='etapa')`	Taxas de conversão em fluxos/etapas de vendas ou processos.
Radar / Polar	`px.line_polar(df, r='v', theta='cat')`	Comparação multi-eixo de perfis e habilidades.
Ternário	`px.scatter_ternary(df, a='a', b='b', c='c')`	Distribuição de proporções entre três componentes.
Choropleth (Mapa)	`px.choropleth(df, locations='iso', color='v')`	Mapas temáticos interativos por regiões/países.
Densidade 2D	`px.density_heatmap(df, x='x', y='y')`	Concentração e agrupamento de pontos numéricos.

7. Módulo OS e Gerenciamento Avançado de Caminhos (Paths)

O módulo nativo **`os`** (Sistema Operacional) permite que o programa interaja com o sistema de arquivos. Ele garante compatibilidade entre Windows (`\`) e Linux/macOS (`/`), prevenindo erros de carregamento.

Função	Descrição e Aplicação Prática
`os.getcwd()`	Retorna o caminho absoluto do diretório de trabalho atual do Python.
`os.chdir("caminho")`	Altera o diretório de trabalho atual para a pasta especificada.
`os.listdir("pasta")`	Lista todos os nomes de arquivos e pastas dentro de um diretório.
`os.path.join(p1, p2)`	Combina caminhos de forma segura e cross-platform.
`os.path.exists("caminho")`	Verifica se um arquivo ou diretório existe (retorna `True` / `False`).
`os.mkdir() / os.makedirs()`	Cria novas pastas na estrutura de diretórios do sistema.

Exemplo Prático de Construção Dinâmica de Caminho:

```python
import os
import pandas as pd

# Define o caminho base de forma compatível com qualquer sistema operacional
pasta_dados = "dados_projeto"
nome_arquivo = "vendas_2026.csv"

caminho_completo = os.path.join(os.getcwd(), pasta_dados, nome_arquivo)

if os.path.exists(caminho_completo):
    df = pd.read_csv(caminho_completo)
    print("Arquivo carregado com sucesso!")
else:
    print(f"Erro: Arquivo não encontrado em {caminho_completo}")
```